

Clasificación multiclase del riesgo de burnout en población Gen Z mediante redes neuronales multicapa

Patricia Rodrigo Barrio y Víctor Rodríguez Albendea
Universitat Politècnica de València, Valencia, España

Resumen

Este trabajo aborda la clasificación de la variable Burnout_Risk (Low, Medium y High) en un conjunto de datos tabular de 10.000 observaciones de población Gen Z. El pipeline incluyó partición estratificada en entrenamiento, validación y test, normalización z-score de variables numéricas, codificación one-hot de variables categóricas y ponderación de clases para compensar el desbalanceo de la categoría Low. Se compararon un baseline lineal, una red shallow, una búsqueda automática con Keras Tuner y varias redes profundas con cambios de activación, dropout y optimizador. Los resultados mostraron una mejora progresiva respecto al baseline (accuracy = 0,901) hasta alcanzar el mejor rendimiento con una red profunda con dropout, activación ReLU y optimizador RMSprop (accuracy = 0,997; F1 macro = 0,998). No obstante, dado el bajo soporte de la clase Low en test ($n = 6$), sus métricas deben interpretarse con cautela.

1. Introducción

El burnout ha adquirido un interés creciente en población joven por su asociación con agotamiento emocional, pérdida de motivación y deterioro del bienestar. En etapas académicas y laborales tempranas, puede verse influido por hábitos digitales intensivos, calidad del sueño, carga de estudio y factores psicológicos como la ansiedad o el overthinking. Desde el punto de vista del análisis de datos, se trata de un problema de clasificación supervisada multiclase sobre variables tabulares heterogéneas.

En este contexto, las redes neuronales multicapa constituyen una alternativa adecuada para modelar relaciones no lineales entre variables de distinta naturaleza. Sin embargo, su aplicación exige un flujo analítico riguroso que incluya partición entrenamiento-validación-test, preprocesado adecuado, selección de hiperparámetros y control del sobreajuste [1]. Además, el fuerte desbalanceo de la clase Low obliga a complementar la accuracy con métricas que reflejen mejor el comportamiento sobre la clase minoritaria. En este trabajo se compararon distintas configuraciones de redes neuronales para clasificar Burnout_Risk, analizando el efecto de la arquitectura, la activación, la regularización y el optimizador sobre el rendimiento global y sobre la recuperación de la clase menos representada.

El objetivo de este trabajo fue comparar distintas configuraciones de redes neuronales multicapa para clasificar la variable Burnout_Risk en las categorías

Low, Medium y High, analizando el efecto de la arquitectura, la función de activación, la regularización y el optimizador sobre el rendimiento global y sobre la recuperación de la clase minoritaria.

2. Materiales y métodos

2.1. Dataset, partición y preprocesado

El estudio se llevó a cabo sobre un conjunto de datos tabular compuesto por 10.000 observaciones y 22 variables, de las cuales una correspondía a la variable objetivo (Burnout_Risk) [2]. Tras excluir dicha variable, el modelo se entrenó con 21 predictores, distribuidos en 17 variables numéricas y 4 variables categóricas (Gender, Country, Working Status y Content Type Preference). La variable respuesta se codificó como un problema de clasificación multiclase con tres categorías: Low = 0, Medium = 1 y High = 2.

La partición se realizó mediante muestreo estratificado en 8000 muestras de entrenamiento, 1000 de validación y 1000 de test. Las variables numéricas se transformaron mediante estandarización z-score y las categóricas mediante codificación one-hot, ajustando ambos transformadores exclusivamente con el conjunto de entrenamiento para evitar fuga de información [1]. Tras el preprocesado, la dimensión final de entrada fue de 34 características. Dado el acusado desbalanceo de la clase Low, se emplearon pesos de clase de 51,28, 0,733 y 0,619 para Low, Medium y High, respectivamente.

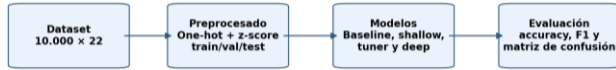


Figura 1. Esquema general del flujo de trabajo seguido: partición estratificada, preprocesado, entrenamiento de modelos, selección en validación y evaluación final en test.

Tabla 1. Resumen del conjunto de datos y del preprocesado.

Característica	Valor
Observaciones totales	10.000
Variables totales	22
Predictores usados	21
Variables numéricas / categóricas	17 / 4
Dimensión final tras el preprocesado	34
Partición train / val / test	8000 / 1000 / 1000
Distribución global de la variable objetivo	Low 64; Medium 4548; High 5388
Pesos de clase (Low / Medium / High)	51,28 / 0,733 / 0,619

2.2. Arquitecturas y configuraciones analizadas

Se evaluaron cinco bloques experimentales con el fin de analizar el efecto de la complejidad arquitectónica, la regularización y las decisiones de optimización sobre el rendimiento de clasificación. En primer lugar, se entrenó un modelo lineal baseline, implementado como una única capa densa de salida con tres neuronas y activación softmax, adecuada para un problema multiclase. A continuación, se estudió una red neuronal shallow, compuesta por una sola capa oculta de 200 neuronas ReLU y una capa de salida softmax, con dos valores de learning rate para analizar la sensibilidad inicial del entrenamiento. En una tercera fase se llevó a cabo una búsqueda automática de hiperparámetros mediante Keras Tuner con estrategia Hyperband [4], utilizando como criterio de selección la accuracy en validación. El espacio de búsqueda incluyó el número de neuronas de la capa oculta (64, 128, 200 y 256), la tasa de aprendizaje (0,01; 0,001; 0,0001) y la función de activación (ReLU, tanh y sigmoid). La arquitectura óptima devuelta por esta búsqueda fue posteriormente evaluada sobre el conjunto de test. En

cuarto lugar, se entrenó una red profunda fully connected con cinco capas ocultas de tamaños 512-256-128-64-32, activación ReLU y optimizador Adam [5], con el objetivo de analizar si un aumento de profundidad permitía mejorar la capacidad de modelado frente a las arquitecturas más simples [1]. Por último, se incorporó un bloque de regularización mediante dropout sobre esta arquitectura profunda, manteniendo la misma secuencia de capas ocultas y evaluando dos intensidades de apagado aleatorio ($p = 0,25$ y $p = 0,50$) [3]. Sobre esta versión regularizada se realizaron, además, dos comparaciones adicionales: una comparación de funciones de activación (ReLU, tanh, sigmoid y ELU) y una comparación de optimizadores (Adam, SGD y RMSprop). En todos los casos, la capa de salida se mantuvo fija con tres neuronas y activación softmax, y el entrenamiento se realizó mediante mini-batches y monitorización del comportamiento en validación, lo que permitió controlar la generalización y detectar posibles patrones de sobreajuste [1].

Tabla 2. Configuraciones experimentales evaluadas.

Bloque	Configuración principal
Baseline lineal	Capa densa única con 3 salidas softmax; SGD; $lr = 0,005$; 50 épocas; batch = 128
Shallow network	1 capa oculta de 200 neuronas ReLU + salida softmax; pruebas con $lr = 0,01$ y $0,001$; 30 épocas
Keras Tuner	Búsqueda Hyperband sobre unidades {64, 128, 200, 256}, lr {0,01; 0,001; 0,0001} y activación {ReLU, tanh, sigmoid}; 30 épocas
Deep network	Arquitectura 512-256-128-64-32; ReLU; Adam; $lr = 0,001$; 50 épocas
Deep + dropout	Misma arquitectura profunda con $dropout$ $p = 0,25$ y $p = 0,50$
Comparación activaciones	ReLU, tanh, sigmoid y ELU sobre la arquitectura profunda con $dropout$
Comparación optimizadores	Adam, SGD y RMSprop sobre la arquitectura profunda con $dropout$

2.3. Métricas de evaluación

La selección de modelos se basó en la accuracy de validación, utilizada tanto para comparar configuraciones como en la búsqueda automática con Keras Tuner. No obstante, debido al desbalanceo entre clases, la evaluación final en test se complementó con precision, recall y F1-score por clase, prestando especial atención a la categoría Low, por ser la menos representada. Además, se empleó la matriz de confusión para analizar los patrones de error entre clases y las curvas de pérdida y accuracy en entrenamiento y validación para detectar posibles fenómenos de sobreajuste o infraajuste [1].

3. Resultados

3.1. Comparación de configuraciones

El modelo baseline lineal alcanzó una accuracy de 0,901 en test. Aunque este resultado ya indicaba una capacidad predictiva razonable, el análisis por clases mostró limitaciones claras: la clase Low presentó un recall de 1,00, pero con una precision de solo 0,10 y un F1-score de 0,18, lo que indica que el modelo tendía a sobredetectar la clase minoritaria a costa de generar numerosos falsos positivos.

La incorporación de no linealidad mediante una red shallow con una capa oculta de 200 neuronas ReLU mejoró de forma apreciable el rendimiento cuando se empleó learning rate = 0,01, alcanzando una accuracy de 0,961. En esta configuración, el F1-score de la clase Low aumentó hasta 0,444, manteniéndose el recall en 1,00. Sin embargo, cuando se redujo la tasa de aprendizaje a 0,001, el rendimiento descendió hasta 0,867, lo que sugiere una convergencia insuficiente bajo el número de épocas fijado.

La búsqueda automática de hiperparámetros mediante Keras Tuner identificó como mejor configuración una red con 64 neuronas, activación sigmoid y learning rate = 0,01, que alcanzó una accuracy de 0,996 en test y un F1-score de 0,857 para la clase Low. Este resultado confirmó que una selección automática de hiperparámetros podía mejorar notablemente el rendimiento respecto a arquitecturas más simples. Por su parte, la red profunda sin dropout (arquitectura 512-256-128-64-32) obtuvo una accuracy de 0,993 en test y mostró el comportamiento más equilibrado entre clases, con

valores prácticamente perfectos en las tres categorías y F1-score = 1,00 en la clase Low. La introducción de dropout moderado ($p = 0,25$) sobre la arquitectura profunda mantuvo un rendimiento muy alto, con una accuracy de 0,995. No obstante, el comportamiento sobre la clase minoritaria fue ligeramente inferior al de la red profunda sin dropout, con F1-score = 0,80 en la categoría Low. Cuando la intensidad de regularización aumentó a $p = 0,50$, el rendimiento global descendió hasta 0,985 y el F1-score de Low cayó a 0,50, lo que sugiere que una regularización excesiva deterioró la capacidad discriminativa del modelo.

La comparación entre funciones de activación sobre la arquitectura profunda regularizada mostró diferencias apreciables. La activación sigmoid alcanzó una accuracy de 0,991 y el mejor resultado dentro de este bloque para la clase Low ($F1 = 0,923$), mientras que tanh y ELU obtuvieron rendimientos algo inferiores (0,986 y 0,989, respectivamente). En conjunto, estas pruebas indican que el efecto de la activación no fue despreciable, especialmente al analizar la estabilidad sobre la clase minoritaria.

En la comparación de optimizadores, los contrastes fueron todavía más marcados. Con SGD, la accuracy descendió hasta 0,823, confirmando una convergencia claramente peor para esta arquitectura profunda. En cambio, Adam mantuvo un rendimiento muy elevado (0,995), mientras que RMSprop ofreció el mejor resultado global del estudio, con una accuracy de 0,997 y métricas prácticamente perfectas en todas las clases.

Tabla 3. Comparación del rendimiento en test de las principales configuraciones evaluadas.

Modelo	Accuracy test	F1-score Low
Baseline lineal	0,901	0,180
Shallow net (ReLU, lr = 0,01)	0,961	0,444
Shallow net (ReLU, lr = 0,001)	0,867	0,136
Keras Tuner (64, sigmoid, lr = 0,01)	0,996	0,857
Deep net	0,993	1,000
Deep + dropout (p = 0,25)	0,995	0,800
Deep + dropout (p = 0,50)	0,985	0,500

Deep + dropout + RMSprop	0,997	1,000
-------------------------------------	--------------	--------------

3.2. Análisis del modelo final

El mejor comportamiento global se obtuvo con la red profunda con dropout $p = 0,25$, activación ReLU y optimizador RMSprop, que alcanzó una accuracy de 0,997 en test. En esta configuración, la clase Low presentó $\text{precision} = 1,00$, $\text{recall} = 1,00$ y $\text{F1-score} = 1,00$, mientras que las clases Medium y High también mostraron valores superiores a 0,996 en F1-score.

La matriz de confusión confirmó este comportamiento casi perfecto. De las 1000 observaciones del conjunto de test, solo se produjeron tres errores de clasificación, todos ellos entre las clases Medium y High, sin errores asociados a la categoría Low. Este resultado sugiere que, dentro del conjunto analizado, la principal dificultad residual del problema no estuvo en detectar la clase minoritaria, sino en discriminar casos próximos entre los niveles intermedios y altos de riesgo.

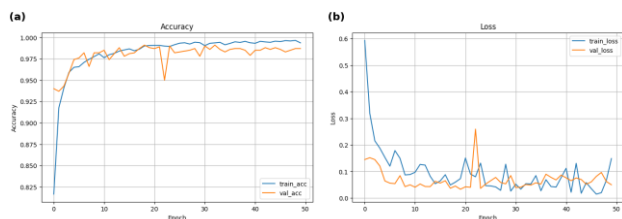


Figura 2. Curvas del mejor modelo (deep + dropout + RMSprop): (a) accuracy y (b) loss.

Tabla 4. Matriz de confusión del mejor modelo en test.

Clase real \ predicha	Low	Medium	High
Low	6	0	0
Medium	0	454	1
High	0	2	537

4. Discusión y conclusiones

Los resultados obtenidos muestran que las redes neuronales multicapa constituyen una herramienta muy eficaz para la clasificación de la variable Burnout_Risk en datos tabulares heterogéneos. Frente al baseline lineal, las arquitecturas no lineales mejoraron de forma consistente tanto la accuracy global como el rendimiento sobre la clase minoritaria, lo que confirma la utilidad de introducir capas ocultas

para capturar relaciones más complejas entre las variables de entrada.

El estudio también puso de manifiesto que no todas las decisiones de diseño afectan del mismo modo al rendimiento. En particular, la elección del learning rate, la regularización mediante dropout y el optimizador tuvo un impacto claro en la estabilidad y la capacidad predictiva del modelo. Una regularización moderada resultó beneficiosa, mientras que una intensidad excesiva redujo la capacidad discriminativa, especialmente sobre la clase Low. Asimismo, RMSprop ofreció el mejor comportamiento global entre los optimizadores evaluados.

Otro aspecto relevante es que, aunque varios modelos alcanzaron $\text{recall} = 1,00$ en la clase Low, solo las configuraciones más avanzadas consiguieron combinar esa sensibilidad con una alta precisión, evitando un número elevado de falsos positivos. Por tanto, la mejora no debe interpretarse únicamente como un incremento de accuracy, sino como una clasificación más equilibrada entre las tres categorías.

En resumen, el mejor modelo fue la red profunda con dropout $p = 0,25$ y optimizador RMSprop, que alcanzó una accuracy de 0,997 y un comportamiento prácticamente perfecto en test. Estos resultados indican que, en este conjunto de datos, una arquitectura MLP adecuadamente ajustada puede ofrecer un rendimiento muy alto. No obstante, la interpretación de estos hallazgos debe realizarse con cautela, especialmente debido al fuerte desbalanceo de la clase minoritaria y a la naturaleza del dataset empleado. Como línea futura, resultaría de interés contrastar este enfoque sobre datos reales y analizar la robustez del modelo tras eliminar variables potencialmente muy próximas semánticamente al target.

Referencias

- [1] MARÍN-MORALES, J. *Fundamentos de las redes neuronales y deep learning*. Material docente de la asignatura *Simulación y Redes Neuronales*. Universitat Politècnica de València, 2026.
- [2] ANSARI, Hammad. *Gen Z Mental Wellness & Digital Lifestyle Patterns*. Kaggle, 2025. Dataset en línea.
- [3] SRIVASTAVA, Nitish, et al. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 2014, vol. 15, no 1, p. 1929-1958.
- [4] CHOLLET, François, et al. keras, GitHub [en línea]. 2015.
- [5] KINGMA, Diederik P.; BA, Jimmy. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.